



LangChain -Chains

By 



LangChain - Chains

In **LangChain**, chains define how multiple steps (Prompts, LLMs, parsers, functions) are executed together.

The three important execution patterns are:

- **Sequential chains**
- **Parallel chains**
- **Conditional chains**



LangChain – Sequential chains

Sequential Chain (Step-by-step execution)

Output of Step 1 → becomes input of Step 2 → becomes input of Step 3 ...

| pipe operator makes it **sequential**.

```
prompt1 = PromptTemplate(
    template='Generate a detailed report on {topic}',
    input_variables=['topic']
)

prompt2 = PromptTemplate(
    template='Generate a 5 pointer summary from the following text \n {text}',
    input_variables=['text']
)

model = ChatGoogleGenerativeAI(model = "gemini-2.5-flash")

parser = StrOutputParser()

chain = prompt1 | model | parser | prompt2 | model | parser

result = chain.invoke({'topic': 'GPU usage in INDIA'})
```



LangChain – Parallel chains

Run multiple steps at same time)

Same input → multiple chains → executed simultaneously → results combined

Will use RunnableParallel

```
from langchain_core.runnables import RunnableParallel

summary_chain = (
    ChatPromptTemplate.from_template("Summarize:\n{text}")
    | model
    | StrOutputParser()
)

sentiment_chain = (
    ChatPromptTemplate.from_template("What is the sentiment?\n{text}")
    | model
    | StrOutputParser()
)

parallel_chain = RunnableParallel(
    summary=summary_chain,
    sentiment=sentiment_chain
)

result = parallel_chain.invoke({"text": "LangChain is powerful and flexible."})
print(result)
```

LangChain – Conditional chains



Conditional Chain (If-Else logic)

Will use RunnableBranch

```
Branch_chain= RunnableBranch(  
    (Condition1, chain1),  
    (Condition2, chain2)  
    default chain  
    )
```

```
from langchain_core.runnables import RunnableBranch
```

```
def is_question(input):  
    return "?" in input["text"]
```

```
question_chain = (  
    ChatPromptTemplate.from_template("Answer this:\n{text}")  
    | model  
    | StrOutputParser()  
    )
```

```
statement_chain = (  
    ChatPromptTemplate.from_template("Summarize this:\n{text}")  
    | model  
    | StrOutputParser()  
    )
```

```
conditional_chain = RunnableBranch(  
    (is_question, question_chain),  
    statement_chain # default  
    )
```

```
result = conditional_chain.invoke({"text": "What is LangChain?"})  
print(result)
```

LangChain – RunnableLambda



Example 1:

```
def preprocess(x):  
    x["text"] = x["text"].lower()  
    return x
```

```
pre = RunnableLambda(preprocess)
```

```
chain = pre | prompt | model | StrOutputParser()
```

Example 2:

```
format_output = RunnableLambda(lambda x: x.strip())
```

```
chain = prompt | model | StrOutputParser() | format_output
```

A **lambda** is an anonymous (unnamed) function.

Syntax: lambda parameters: expression

- Only **one expression**
- No multiple lines
- Automatically returns the expression

```
def check_positive(x):  
    return x.sentiment == "positive"
```

= lambda x: x.sentiment == "positive"

```
def add_one(x):  
    return x + 1
```

= lambda x: x + 1

Thank you